TD3 — Communication : la clef pour avoir des programmes heureux !

Consignes générales

Chaque exercice décrit un besoin. Il vous est demandé d'en proposer une modélisation sous forme de **diagramme de classes UML**, puis de **justifier vos choix** en quelques lignes.

Dans chaque exercice, une ou plusieurs questions portent sur une *évolution* du besoin. Traitez-les sérieusement : **la qualité d'une conception se mesure à ce qu'elle coûte quand le cahier des charges change.**

Exercice 1 — Une agence de vélos

Une agence de location de vélos en libre-service souhaite informatiser la gestion de sa flotte.

— Un vélo est identifié par un numéro, une date de mise en service et un kilométrage. L'agence loue des vélos mécaniques et des vélos électriques ; ces derniers portent une batterie, dont on connaît le niveau de charge et le nombre de cycles.

— Une station a un nom, une adresse et un nombre de bornes. Un vélo est soit rangé à une borne, soit en location, soit à l'atelier.

— Un client est identifié par un numéro, un nom et une date d'inscription. Il souscrit un forfait : à la course, à la journée, ou à l'année.

— Une location porte sur un vélo et un client, part d'une station et se termine à une station, qui peut être la même. On en connaît l'heure de début et l'heure de fin.

— Un client ne peut avoir qu'une location en cours à la fois. Un vélo à l'atelier ne peut pas être loué.

— Le prix d'une location dépend du forfait du client et de la durée : la première demi-heure est comprise, puis chaque demi-heure entamée est facturée, selon un tarif propre au forfait.

— Une intervention d'atelier porte sur un vélo, a une date, un motif et un technicien. Un vélo peut avoir subi un nombre quelconque d'interventions.

Q1. Proposez un diagramme de classes modélisant ce domaine : attributs, associations, multiplicités, classes abstraites.

Q2. Où est rangée l'information « ce vélo est actuellement à telle borne » ? Discutez au moins deux emplacements possibles et leurs conséquences.

Q3. Parmi les règles de l'énoncé, certaines s'expriment par une multiplicité, d'autres non. Classez-les, et proposez une manière de dire celles qui restent.

Q4. L'agence ouvre un service de trottinettes, puis des stations sans borne où le véhicule est simplement cadenassé. Faites évoluer votre diagramme : quelles classes écrivez-vous, lesquelles modifiez-vous ?

Exercice 2 — Le circuit d'un manuscrit

On s'intéresse maintenant au circuit éditorial de la maison d'édition de l'exercice 2 du TD 2. Un manuscrit avance par étapes : il est d'abord *soumis*, puis *en relecture*, puis *accepté* ou *refusé* ; un manuscrit accepté passe *en production*, et finit *publié*.

Il faut permettre à un manuscrit de progresser dans la chaîne d'édition. Le traitement que subit le manuscrit dépend de l'étape. Un manuscrit soumis est confié à un relecteur. En relecture, le relecteur rend son avis : s'il accepte, on cherche un correcteur, sinon le manuscrit est refusé. Une fois accepté, on cherche un imprimeur et on fixe le volume du tirage. En production, le manuscrit est publié. Publié ou refusé, il n'avance plus. Votre solution doit de plus informer le client si le manuscrit a pu passer à l'étape suivante ou non.

Chaque étape a donc besoin d'interlocuteurs pour répondre à ses questions : qui relit, qui corrige, qui imprime, en quel volume.

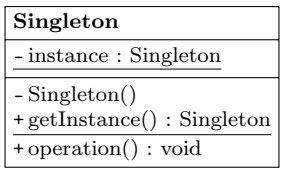
Le circuit sera emmené à évoluer : l'éditeur peux ajouter des étapes. Cet ajout ne doit obliger à modifier ni le manuscrit, ni aucune autre classe ou interface que celle qui mène désormais à la nouvelle étape. Enfin, aucune classe de votre modèle ne doit comporter de test sur l'étape courante d'un manuscrit.

Q1. Proposez un diagramme de classes répondant à ce besoin.

Q2. Où, dans votre modèle, est-il écrit qu'un manuscrit publié n'avance plus ?

Q3. L'éditeur intercale une étape *en traduction*, entre l'acceptation et la production : il faut alors un traducteur et une langue cible. Quelles classes écrivez-vous, lesquelles modifiez-vous ?

Seconde partie. Voici un patron de conception.



Les membres soulignés appartiennent à la classe et non aux instances. On impose maintenant la contrainte suivante : pour chaque besoin, il n'existe qu'un seul vivier, partagé par tous les manuscrits de la maison — un vivier de relecteurs, un de correcteurs, un d'imprimeurs.

Q4. Comment, en utilisant le patron ci-dessus, codez-vous cette contrainte ? Comment cela s'incorpore-t-il dans votre diagramme précédent ?

Q5. Une classe dont tous les membres seraient statiques conviendrait-elle ici ? Justifiez.

Troisième partie. Il reste à dire comment un vivier désigne l'interlocuteur. Les inscrits d'un vivier ne sont pas connus à la conception : on s'y inscrit et on s'en retire à tout moment, et leur nombre n'est pas borné. Par exemple, lorsque l'étape *soumis* demande un relecteur, tous les inscrits du vivier sont prévenus ; chacun répond s'il est intéressé par ce manuscrit ; l'un des intéressés est ensuite désigné. Si personne ne se déclare, le manuscrit n'avance pas.

On suppose disposer d'un mécanisme qui, dès lors qu'un acteur humain (relecteur, correcteur, imprimeur, ...) est appelé, suspend le programme jusqu'à sa réponse ; sa réalisation n'est pas l'objet de l'exercice.

Q6. Faites évoluer votre diagramme pour rendre compte de cette désignation.

Q7. Que se passe-t-il si personne ne se déclare ? Si plusieurs se déclarent ? Où ces deux règles sont-elles écrites dans votre modèle ?

Q8. Comparez ce mécanisme à celui des soins du fermier (TD2, exercice 4). Dans les deux cas, un objet s'adresse à une liste d'objets qu'il ne connaît pas. Qu'est-ce qui change ?

Exercice 3 — Exporter un document

La maison d'édition exporte ses documents vers l'extérieur. Un export part d'un document et produit un fichier, mais il peut subir plusieurs traitements en chemin : compression, chiffrement, signature électronique, horodatage.

Ces traitements se combinent librement : un export peut n'en subir aucun, un seul, ou plusieurs. Leur ordre compte — compresser puis chiffrer ne donne pas le même fichier que chiffrer puis compresser — et un même traitement peut apparaître deux fois. L'éditeur peux en ajouter d'autres, et cet ajout ne doit modifier aucune classe existante. Enfin, le code qui demande un export ne doit pas avoir à savoir combien de traitements ont été appliqués, ni lesquels : il manipule un export, un point c'est tout.

Q1. Proposez un diagramme de classes répondant à ce besoin.

Q2. Pourquoi une liste d'options — un booléen par traitement, ou un ensemble de traitements à appliquer — ne convient-elle pas ?

Q3. L'éditeur veut connaître la taille du fichier produit à chaque étape du traitement. Votre modèle le permet-il ?

Q4. Comparez votre diagramme à celui du catalogue de l'éditeur (TD2, exercice 2). Ils se ressemblent beaucoup. Qu'est-ce qui les distingue ?